

Esame OS Internals (Cabodi)

Data: 22/01/2024

Nota Generale: In tutte le domande, ogni volta che il risultato è una selezione, un SI/NO, o un numero, sono necessari una motivazione/spiegazione o passaggi intermedi che portino al risultato.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e Page Fault)

Contesto: Nel contesto della gestione della memoria virtuale, si consideri un sistema con paginazione:

- Definire che cos'è un page fault, spiegare le condizioni in cui si verifica un page fault e le conseguenze per il sistema operativo.
- Elencare i passi necessari per gestire un page fault, riferendosi in particolare al modo in cui il sistema operativo risponde a un page fault e il meccanismo utilizzato per risolverlo.
- Si consideri la matrice bidimensionale A: `int A[100][100]`, dove `A[0][0]` si trova all'indirizzo 800 in un sistema di memoria indirizzabile a Byte, gestita con paginazione e pagine di dimensioni pari a 800 Byte. La dimensione di un numero intero è di 32 bit. Il processo che manipola la matrice risiede a pagina 0 (posizioni da 0 a 799). Pertanto, ogni fetch di istruzioni avverrà alla pagina 0. I frame disponibili sono tre.

Quanti page fault vengono generati dal seguente ciclo di inizializzazione della matrice? Utilizzare la politica di sostituzione pagine LRU e supporre che un frame contenga il codice e che gli altri due siano inizialmente vuoti.

```
1. for (int j=0; j<100; j++)
    for (int i=0; i<100; i++){
        A[i][j]=0;
    }

2. for (int i=0; i<100; i++)
    for (int j=0; j<100; j++){
        A[i][j]=0;
    }
```

Domanda 2 (RAID e Affidabilità)

- A) Spiegare la struttura RAID, discuterne i vantaggi, spiegando come il livello RAID scelto migliori l'archiviazione e il recupero dei dati rispetto a un singolo disco.

- A) Nel contesto della struttura RAID, definire il tempo medio di guasto (MTTF), il tempo medio di perdita di dati (MTTDL) e il tempo medio di riparazione (MTTR). Fornire una spiegazione di ciascun termine ed elaborare il modo in cui queste metriche contribuiscono all'affidabilità e alla disponibilità complessive di un sistema RAID.
 - A) Prendere in considerazione una struttura RAID con 4 dischi in configurazione con mirroring (RAID 1). Supponendo che ogni disco possa guastarsi indipendentemente dagli altri, con un MTTF di 50000 ore e un MTTR di 10 ore, calcolare l'MTTDL. Fornire una spiegazione del processo di calcolo.
-

Domanda 3 (Gestione Spazio Libero: Liste vs Bitmap)

Contesto: Si consideri l'utilizzo di liste concatenate e bitmap per la gestione dello spazio libero, sia per i free frame (nella gestione della memoria) che per i blocchi liberi (nei file system). Rispondere sì/no (vero/falso) alle seguenti affermazioni e fornire la motivazione della risposta:

- A) Le bitmap sono migliori delle liste concatenate quando si cercano intervalli di frame/blocchi contigui.
- A) In un sistema con paginazione, le free list sono la soluzione migliore sia per la memoria user (processo) che kernel.
- A) I blocchi liberi per un file system basato su i-node sono gestiti meglio con le bitmap.
- A) La ricerca di un blocco libero in una free list (concatenata) ha un costo lineare (nella dimensione della lista) nella gestione della memoria basata su una page table a due livelli.
- A) Un free list (concatenata) ordinata ha un costo lineare (nella dimensione della lista) sia per l'allocazione che per la liberazione dello spazio contiguo.
- A) Una bitmap, sebbene richieda memoria aggiuntiva, fornisce prestazioni in termini di tempo migliori (di una free list) in un file system basato su FAT (File Allocation Table).

2 SECTION 2: OS161

Domanda 4 (Chiamata di Sistema `read()`)

Contesto: Si consideri una possibile implementazione della chiamata di sistema `read()` in OS161.

- B) La destinazione della `read` può essere nella memoria kernel o deve essere (sempre) nella memoria user?
- B) Supponiamo che due processi user, in esecuzione contemporaneamente, aprano lo stesso file per `read` e chiamino `read(fd, v, N)`, su quel file, ci si aspetta che (per ogni opzione rispondere SI/NO e motivare):
1. `fd` è lo stesso (numero intero) per entrambi i processi.
 2. I due processi leggono il file indipendentemente l'uno dall'altro (o condividono lo stesso flusso/sequenza di lettura?).
 3. È necessario un blocco, perché le operazioni devono essere eseguite in mutua esclusione.
 4. Se l'implementazione della chiamata di sistema `read` utilizza un buffer kernel, i due processi utilizzeranno lo stesso buffer.
- B) Tenuto conto della seguente implementazione del `sys_read`, per letture su file:

```
static int
file_read (int fd, userptr_t buf_ptr, size_t size) {
    kbuf = kmalloc(size);
    uio_kinit(&iiov, &ku, kbuf, size, of->offset, UIO_READ);
    result = VOP_READ (vn, &ku);

    if (result) {
        return result;
    }

    of->offset = ku.uio_offset;
    nread = size - ku.uio_resid;
    copyout (kbuf, buf_ptr, nread);
    kfree (kbuf);

    return (nread);
}
```

Rispondere alle seguenti domande relative al codice:

1. L'operazione sfrutta un buffer del kernel? (motivare)
 2. È possibile che il valore ritornato sia diverso dal valore del parametro `size`? (motivare)
 3. `kmalloc` può essere chiamato con un parametro `< size` (ad esempio `kmalloc(size/2)`)? (motivare)
-

Domanda 5 (Gestione Thread)

Contesto: E' dato un sistema OS161.

- B) Considerare le funzioni `thread_switch` e `thread_yield`. Sono equivalenti o svolgono compiti diversi? (motivare)
- B) Quando viene creato un nuovo thread del kernel in OS161, il thread viene eseguito immediatamente o viene accodato per una successiva esecuzione?
- B) L'implementazione di `thread_exit()` termina con una chiamata:
`switchframe_switch(&cur->t_context, &next->t_context);`
Cosa rappresentano i due parametri (non basta dire "contesto del thread")? Quale dei due parametri si riferisce al contesto salvato e quale al contesto ripristinato?

SOLUZIONI

Domanda 1 (Memoria Virtuale e Page Fault)

A) Definizione e conseguenze del Page Fault:

Un page fault è un'eccezione hardware (trap) generata dalla Memory Management Unit (MMU) quando la CPU tenta di accedere a un indirizzo virtuale la cui pagina corrispondente ha il *valid-invalid bit* impostato su "invalid" nella Page Table (ovvero, la pagina non è attualmente caricata in un frame fisico della RAM). Le conseguenze per il Sistema Operativo consistono nell'interrompere l'esecuzione del processo corrente (passaggio in kernel mode), salvare il suo stato e avviare le routine di gestione della memoria per recuperare la pagina mancante dalla memoria secondaria (disco/swap).

B) Passi necessari per gestire un page fault:

Il sistema operativo esegue i seguenti step in sequenza:

1. **Trap al SO:** L'hardware salva i registri del processo e cede il controllo al Kernel.
2. **Verifica Validità:** Il SO controlla la tabella interna del processo (PCB) per determinare se il riferimento alla memoria era valido (ma la pagina è su disco) o illegale (accesso fuori dai limiti, che causa l'uccisione del processo tramite *segmentation fault*).
3. **Ricerca Frame:** Se valido, il SO cerca un frame fisico libero. Se non ci sono frame liberi, invoca un algoritmo di sostituzione (es. LRU, FIFO) per selezionare una "pagina vittima" (eventualmente scrivendola su disco se sporca).
4. **Lettura I/O:** Il SO programma un'operazione di lettura dal disco per copiare la pagina richiesta nel frame allocato. Durante l'I/O, il processo è in stato di *wait/blocked* e la CPU viene assegnata ad altri processi.
5. **Aggiornamento Tabelle:** Completato l'I/O, il SO aggiorna la Page Table del processo impostando il nuovo numero di frame e settando il bit a "valid".
6. **Restart dell'Istruzione:** Il processo viene riportato in *ready queue*. Quando ricalcolato, la CPU *riavvia esattamente l'istruzione* che aveva causato il fault, la quale ora andrà a buon fine (page hit).

C) Calcolo Page Fault su Inizializzazione Matrice:

Analisi della struttura dati in memoria:

- La matrice di interi a 32 bit (4 Byte) ha dimensioni 100×100 , occupando 40.000 Byte.
- Una pagina è di 800 Byte. Pertanto occorrono $40.000/800 = 50$ pagine logiche per i dati.
- Capacità pagina: $800/4 = 200$ interi. Poiché in C le matrici sono memorizzate per riga, ogni singola pagina contiene esattamente **2 righe intere** della matrice ($100 + 100$ interi).
- **Vincoli Hardware:** Il sistema ha solo 3 frame fisici. Uno è stabilmente occupato dalla Pagina 0 (codice). Restano **soltanto 2 frame** a disposizione per caricare i dati della matrice.

Calcolo Ciclo 1 (Accesso per Colonna - j esterno):

Il ciclo scorre la matrice verticalmente (es. $A[0][0]$, $A[1][0]$, $A[2][0]$...).

- $A[0][0]$ e $A[1][0]$ risiedono nella 1^a pagina (1 fault, 1 hit).
- $A[2][0]$ e $A[3][0]$ risiedono nella 2^a pagina (1 fault, 1 hit).

- Per completare una singola colonna, il programma deve scorrere tutte le 50 pagine. Avendo solo 2 frame fisici per i dati e politica LRU, il sistema soffre di *thrashing*: quando serve la 3^a pagina, la 1^a viene sfrattata. Al passaggio alla colonna successiva ($j = 1$), nessuna delle pagine necessarie sarà più in RAM.
- Risultato: 50 fault per colonna. $50 \text{ fault} \times 100 \text{ colonne} = \mathbf{5000 \text{ page fault}}$.

Calcolo Ciclo 2 (Accesso per Riga - i esterno):

Il ciclo scorre la matrice orizzontalmente (es. $A[0][0]$ fino a $A[0][99]$).

- La prima riga intera è interamente contenuta nella 1^a pagina. Provoca 1 fault iniziale e 99 hit.
- La seconda riga è ancora contenuta nella 1^a pagina. Provoca 100 hit.
- La 1^a pagina viene abbandonata solo dopo aver esplorato 200 interi sequenziali. Grazie all'ottima località spaziale, ogni pagina viene caricata in memoria una sola volta, nonostante la limitazione dei 2 frame.
- Risultato: 1 fault per pagina. $1 \text{ fault} \times 50 \text{ pagine} = \mathbf{50 \text{ page fault}}$.

Domanda 2 (RAID e Affidabilità)

A) Spiegazione della struttura RAID:

Le architetture RAID (Redundant Array of Independent Disks) combinano più dischi fisici in una singola unità logica per migliorare le prestazioni (efficienza) e/o la ridondanza (affidabilità). I tre meccanismi fondamentali sono:

- **Mirroring (es. RAID 1):** Consiste nella replica esatta dei dati su più dischi. Massimizza l'affidabilità poiché il sistema sopravvive finché almeno un disco della copia è integro.
- **Striping (es. RAID 0):** Consiste nella suddivisione dei dati in blocchi (stripes) distribuiti su più dischi. Massimizza l'efficienza (velocità I/O parallela) ma non offre alcuna ridondanza.
- **Parità (es. RAID 5):** Utilizza operazioni logiche (XOR) per calcolare bit di controllo distribuiti. Offre un compromesso eccellente: permette l'error correction (ricostruzione dei dati di un disco guasto) sprecando meno spazio rispetto al mirroring.

B) Metriche di Affidabilità (MTTF, MTDDL, MTTR):

- **MTTF (Mean Time To Failure):** È il tempo medio atteso prima che un singolo dispositivo subisca un guasto hardware.
- **MTTR (Mean Time To Repair):** È il tempo medio necessario per sostituire il dispositivo guasto e ricostruire completamente i dati su di esso.
- **MTDDL (Mean Time To Data Loss):** È il tempo medio prima che l'intero sistema RAID perda definitivamente i dati. Si verifica quando si subisce un guasto critico simultaneo (es. la rottura di un disco di mirroring *durante* il tempo di MTTR del suo gemello).

Impatto su Affidabilità e Disponibilità: L'affidabilità del sistema è definita dall'MTDDL. La disponibilità operativa si calcola come $\text{MTTF} / (\text{MTTF} + \text{MTTR})$. I sistemi RAID aumentano drasticamente l'MTDDL e permettono riparazioni "a caldo" per minimizzare l'MTTR, mantenendo alta la disponibilità.

C) Calcolo MTDDL per RAID 1 (4 dischi):

Dati: MTTF = 50.000 ore, MTTR = 10 ore.

In una configurazione RAID 1 standard a 4 dischi, si formano 2 coppie indipendenti di dischi in mirroring.

1. **MTDDL di una singola coppia:** Il sistema perde dati se il secondo disco si guasta durante il tempo di riparazione del primo. La formula (Silberschatz) è:

$$MTDDL_{coppia} = \frac{MTTF^2}{2 \times MTTR} = \frac{(50000)^2}{20} = 125.000.000 \text{ ore}$$

2. **MTDDL del sistema completo:** Poiché il sistema fallisce se *una qualsiasi* delle 2 coppie perde i dati, il tasso di guasto raddoppia, dimezzando il tempo atteso:

$$MTDDL_{sistema} = \frac{MTDDL_{coppia}}{2} = \mathbf{62.500.000 \text{ ore}}$$

Domanda 3 (Gestione Spazio Libero: Liste vs Bitmap)

Risposte e Motivazioni Teoriche (Silberschatz):

1. **Le bitmap sono migliori delle liste concatenate quando si cercano intervalli contigui?**

VERO. La bitmap rappresenta la memoria fisicamente contigua come bit adiacenti. Trovare blocchi contigui richiede solo banali e velocissime operazioni bit-a-bit (ricerca di array di zero). In una free list, i blocchi contigui in memoria non sono necessariamente adiacenti nella lista; trovarli richiederebbe di ordinare e scansionare l'intera struttura, un'operazione estremamente costosa.

2. **In un sistema con paginazione, le free list sono la soluzione migliore sia per user che per kernel?**

FALSO. Sebbene siano eccellenti per i processi utente (dove la MMU risolve la frammentazione e basta estrarre la testa della lista in tempo $O(1)$), sono pessime per il kernel. Quest'ultimo necessita spesso di frame fisicamente contigui (es. per dispositivi hardware o DMA), requisito che una free list non può garantire efficientemente. Per il kernel si usano allocatori specifici come il Buddy System o lo Slab Allocator.

3. **I blocchi liberi per un FS basato su i-node sono gestiti meglio con le bitmap?**

VERO. I file system a i-node beneficiano enormemente della *località spaziale* per ridurre il tempo di seek meccanico del disco. Le bitmap permettono di allocare i blocchi dati di un file nelle immediate vicinanze fisiche del suo i-node. Una free list restituirebbe blocchi casuali sparsi per il disco, frammentando pesantemente il file.

4. **La ricerca in free list ha costo lineare con una page table a due livelli?**

FALSO. Il tipo di Page Table (a 1, 2 o più livelli) è irrilevante per il gestore dello spazio libero. L'estrazione di un frame libero da una classica free list non ordinata ha sempre complessità costante $O(1)$, in quanto consiste nel semplice prelievo del primo elemento in testa alla lista (pop).

5. **Una free list ordinata ha costo lineare per allocazione e liberazione di spazio contiguo?**

VERO. Per l'allocazione, è necessario scorrere la lista linearmente per trovare il primo o il miglior intervallo che soddisfi la richiesta (First-Fit o Best-Fit). Per la deallocazione, bisogna scorrere la lista per trovare la posizione ordinata (per indirizzo fisico) corretta in cui inserire il blocco appena liberato, al fine di poterlo eventualmente fondere (coalescing) con blocchi adiacenti vuoti.

6. Una bitmap fornisce prestazioni migliori di una free list in un FS basato su FAT?

FALSO. In un sistema FAT (File Allocation Table), le informazioni sullo stato dei blocchi (allocati o liberi) sono già integrate strutturalmente all'interno della tabella FAT stessa (es. una entry a 0 indica blocco libero). L'introduzione di una bitmap sarebbe completamente ridondante e non offrirebbe vantaggi prestazionali, costituendo unicamente uno spreco aggiuntivo di memoria.

Domanda 4 (Chiamata di Sistema `read()`)

A) Destinazione della `read`:

La destinazione finale deve essere (sempre) **nella memoria user**. Lo scopo della system call `read` è caricare i dati dal file nello spazio di indirizzamento del processo chiamante (all'indirizzo `buf_ptr`). Poiché il kernel non si fida dei puntatori utente, l'operazione non avviene tramite dereferenziazione diretta, ma attraverso la funzione sicura `copyout()`.

B) Accesso Concorrente (due processi aprono lo stesso file):

1. **Il fd è lo stesso? NO.** La tabella dei file descriptor (FDT) è una struttura privata per ogni processo. L'indice assegnato dipende dallo storico delle aperture di ciascun processo.
2. **Leggono indipendentemente? SI.** Le due chiamate ad `open()` creano due entry distinte nella tabella globale dei file aperti (Open File Table). Ogni processo possiede un proprio `offset` (cursore) indipendente che non interferisce con quello dell'altro.
3. **È necessario un blocco di mutua esclusione? NO.** Entrambi i processi stanno eseguendo operazioni di sola lettura (Read-Only). Le letture non alterano i dati del *inode* (file su disco), pertanto non generano inconsistenze e non richiedono lock sui dati del file.
4. **Utilizzano lo stesso buffer kernel? NO.** Il buffer viene allocato dinamicamente (sulla heap del kernel) all'interno dello scope della funzione `sys_read`. Essendo due processi distinti che eseguono due invocazioni distinte, ognuno otterrà un proprio buffer indipendente in memoria kernel.

C) Analisi Codice `sys_read`:

1. **L'operazione sfrutta un buffer del kernel? SI.** È evidente dalla riga `kbuf = kmalloc(size);`, che alloca un'area di memoria dinamica direttamente all'interno dello spazio di indirizzamento protetto del kernel prima di procedere alla lettura dal file system.
2. **Il valore ritornato può essere diverso dal parametro `size`? SI.** Questo si verifica comunemente quando il file system raggiunge la *End Of File (EOF)* prima di aver letto `size` byte. La variabile `ku.uio_resid` manterrà traccia dei byte "non letti", e `nread` rifletterà l'ammontare effettivamente estratto dal disco.
3. **`kmalloc` può essere chiamato con un parametro `< size`? SI.** Questa è la pratica standard per sistemi operativi robusti. Se un processo malevolo o distratto richiede di leggere svariati Megabyte in un colpo solo, un `kmalloc(size)` esaurirebbe la RAM fisica mandando il sistema in Kernel Panic. L'implementazione corretta prevede di allocare un buffer di dimensione limitata (es. 4 KB) e implementare un ciclo: leggere un chunk, fare `copyout`, avanzare l'offset e ripetere fino ad esaurimento di `size`.

Domanda 5 (Gestione Thread)

A) Funzioni `thread_switch` e `thread_yield`:

Svolgono compiti **diversi ma complementari** e operano a livelli di astrazione differenti.

- **`thread_yield`:** È una funzione di alto livello che gestisce la logica di scheduling. Viene invocata quando un thread cede la CPU (volontariamente o per prelazione). Si occupa di cambiare lo stato del thread da *RUNNING* a *READY*, accodarlo nella *run queue* e selezionare il prossimo thread da eseguire. Al termine di queste operazioni logiche, invoca `thread_switch`.
- **`thread_switch`:** È la funzione di basso livello che implementa il meccanismo vero e proprio. Si occupa di preparare la transizione e invocare la routine assembly (`switchframe_switch`) che effettua lo scambio fisico dei registri sul processore.

B) Esecuzione di un nuovo thread del kernel:

Quando viene creato un nuovo thread (tramite `thread_fork`), questo **non viene eseguito immediatamente**. La fase finale della sua creazione prevede la chiamata a `thread_make_runnable()`, la quale inserisce il nuovo thread nella coda dei processi pronti all'esecuzione (*ready queue*). Il thread attenderà lì finché lo scheduler non lo selezionerà secondo la politica adottata (es. Round Robin).

C) Parametri di `switchframe_switch` in `thread_exit()`:

- **Cosa rappresentano:** I due parametri sono puntatori a strutture dati di tipo `switchframe`. Uno `switchframe` contiene l'esatto stato architetturale della CPU nel momento della sospensione, ovvero i registri hardware fondamentali per la ripresa dell'esecuzione (nello specifico dell'architettura MIPS: lo *Stack Pointer*, il *Return Address* e i registri *callee-save*).
- **Salvato vs Ripristinato:**
 - `&cur->t_context` (primo parametro): Indica l'indirizzo di memoria in cui deve essere **salvato** lo stato corrente della CPU. (*Nota: trovandosi in `thread_exit`, questo salvataggio è fittizio, poiché il thread sta terminando e questo stato non verrà mai più ripristinato*).
 - `&next->t_context` (secondo parametro): Indica l'indirizzo di memoria da cui deve essere **ripristinato** lo stato della CPU, caricando i registri del nuovo thread che sta per entrare in esecuzione.